

Machine Learning

Cybersecurity Project

Part 2: Generative and Discriminative Classification

Module leader: Prof Mounir Ghogho
Lab instructor: Dr Abdelghani Ghanem

Overview

In Lab 1, you audited CICIDS2017 and created evaluation protocols. In this lab, you will compare Naive Bayes classifiers with logistic regression and a single decision tree for binary detection and multiclass traffic classification.

When are generative classifiers competitive with discriminative classifiers, and how do feature representation, modeling assumptions, training-set size, and class priors affect their performance?

The generative models are Gaussian Naive Bayes, Multinomial Naive Bayes, and a hybrid Naive Bayes model. Lab 3 will extend the discriminative comparison to random forests and boosting trees.

Dataset

Download `MachineLearningCSV.zip` from the official CICIDS2017 page:

<https://www.unb.ca/cic/datasets/ids-2017.html>

Lab 1 required `GeneratedLabelledFlows.zip` because its identifiers and timestamps were needed to study grouped and chronological protocols. Lab 2 uses a random split and excludes those variables, so the reduced machine-learning files provide the required flow features and labels without unnecessary metadata. Concatenate the supplied CSV files and retain the original filename only for data auditing, not as a predictor.

Modeling routes

The unknown true posterior is $P(Y = c \mid \mathbf{X} = \mathbf{x})$. A discriminative model approximates it directly with $p_{\theta,c}(\mathbf{x})$ and produces $p_{\hat{\theta},c}(\mathbf{x})$ after training.

A generative classifier estimates a class prior and a class-conditional feature model. Bayes' rule then produces the posterior probability.

Required evaluation protocol

After concatenating the reduced files, assign each row a stable identifier. Check for exact duplicate feature rows before splitting; remove repeated copies with the same label and report any identical feature rows with conflicting labels. Create one fixed stratified random-row training, validation, and test split using the original multiclass traffic label for stratification. Save its row identifiers and random seed, and reuse it for every binary and multiclass model in this lab. Derive the binary label only after splitting. Report the split sizes and class counts.

This protocol evaluates *in-distribution generalization*: performance on new flows drawn from approximately the same mixture of hosts, traffic conditions, and scheduled attack scenarios as the training data. It does not establish generalization to later traffic, new hosts or organizations, or new attack executions and mechanisms.

The random split is used here because CICIDS2017 contains relatively few scheduled attack episodes, and attack categories are strongly tied to particular days and intervals. Episode-level or chronological partitions can therefore leave some attack categories absent from training and confound a comparison of classifier families with changes in attack coverage. Lab 1 examined these more demanding protocols; the purpose here is a controlled comparison of generative and discriminative classifiers on identical data.

Fit every preprocessing step on training data only. Use validation data for the model choices below and evaluate the selected pipelines once on test data.

Part 1: Inspect the numerical features

Before selecting features or fitting a model, screen the candidate numerical features using the training data only. Perform the class-conditional checks separately for benign and attack traffic.

1. Identify non-finite values, constant features, and features with zero or very small within-class variance. State the criterion used to flag a variance as very small.
2. For representative features, inspect class-conditional histograms.
3. For a strongly skewed nonnegative feature, compare its distribution before and after the transformation $\log(1 + x)$. Keep the transformation when it makes the distribution less skewed and easier to model.
4. Remove features that are constant in the training data. Flag the remaining features with very small within-class variance because Gaussian Naive Bayes may require variance smoothing for them.

Exact Gaussianity is not required. The purpose is to identify clear problems before fitting Gaussian Naive Bayes. Provide a compact table containing only the features that require attention, the observed issue, and the action taken: transform, remove, or retain with possible variance smoothing.

Part 2: Three feature representations

Construct the following three representations directly from the reduced files.

A. Numerical subset

Using the diagnostics from Part 1, select 6 to 12 behavior-oriented numerical features. Replace non-finite values using training data, apply the selected transformations, and standardize the features. Use this representation with Gaussian Naive Bayes, logistic regression, and the decision tree.

In this lab, integer-valued packet counts, flag counts, and binary indicators are numerical features; an integer storage type does not by itself make a variable categorical. The exception is **Destination Port**: its values identify services, and numerical distance between port numbers is not meaningful.

B. Destination-port representation

The reduced files contain one native categorical predictor, **Destination Port**. Treat it as a service identifier, not as a continuous numerical quantity. One-hot encode its observed training values and reserve an “other” level for ports not observed in training. Use this port-only representation with Multinomial Naive Bayes, logistic regression, and the decision tree.

C. Complete eligible feature set

Use all eligible behavioral flow features in the reduced files together with categorical **Destination Port**. Remove the repeated **Fwd Header Length.1** column and any feature that is constant in the training set. Do not use filenames or row identifiers as predictors.

Fit hybrid Naive Bayes by using Gaussian class-conditional models for numerical features and smoothed categorical probabilities for categorical features. For each class, begin with its log-prior once and add the log-likelihood contribution of every feature. Normalize the class scores to obtain posterior probabilities.

Create an “other” level during training preprocessing. Map a validation or test port not observed in training to this level, and include it when applying additive smoothing.

Logistic regression uses standardized numerical features and one-hot encoded categorical features. The decision tree uses the same underlying predictors, with one-hot encoded categories but no required numerical scaling.

Required output. Provide one table describing the features, transformations, and final dimension of each representation.

Part 3: Fit and compare the classifiers

Run the following comparisons on identical rows:

Representation	Generative model	Discriminative models
Numerical subset	Gaussian Naive Bayes	Logistic regression and decision tree
Destination port	Multinomial Naive Bayes	Logistic regression and decision tree
Complete eligible set	Hybrid Naive Bayes	Logistic regression and decision tree

Use empirical class priors and begin with the standard Naive Bayes settings. Use the small-variance flags from Part 1. If these features cause unstable estimates, increase the Gaussian variance-smoothing parameter and select its value using validation log loss. Begin with additive smoothing $\alpha = 1$ for Multinomial Naive Bayes and for the categorical part of hybrid Naive Bayes; if stronger or weaker smoothing is needed for rare ports, select α from a small predeclared set using validation log loss. Report every value tested and the selected setting. Never use test results to choose a regularization parameter.

Fit logistic regression without regularization. If it fails to converge or produces unusually large coefficients, refit with L_2 regularization and document the change.

For the decision tree, choose the maximum depth from $\{3, 5, 10, \text{unlimited}\}$ using validation balanced accuracy. Report the selected depth and the first two levels of the fitted tree.

For every model–representation pair, report balanced accuracy, attack precision, attack recall, attack F_1 , test log loss, and a confusion matrix. Organize the results by representation so that each generative model is compared with discriminative models using the same information.

Part 4: Assumption checks

Using training data only:

1. For the numerical subset, show a within-class correlation matrix and identify one clear violation of conditional independence.
2. For the destination-port representation, tabulate the most frequent ports by class and identify rare ports affected by additive smoothing.
3. For the complete feature set, identify important numerical and categorical contributions for two correct and two incorrect hybrid Naive Bayes predictions. State one dependence that the hybrid model ignores.

The assumptions need not be exactly true for classification to be useful. Relate visible violations to the test results.

Part 5: Limited attack data and prior shift

Limited training data

Using the complete representation, form a smaller training set containing 20% of the benign and 20% of the attack examples. Use stratified sampling and a fixed seed, then refit hybrid Naive Bayes, logistic regression, and the selected decision tree. Evaluate on the unchanged test set and identify the model most affected by the smaller minority count.

Different deployment prior

Return to the full training set and suppose the deployment attack prevalence is $\pi_d = 0.05$. Assume that the feature distribution within each class remains unchanged. For hybrid Naive Bayes, keep the class-conditional model fixed and replace the training prior.

For logistic regression and the decision tree, let $r_t(\mathbf{x})$ be the fitted attack posterior under training prior π_t . Correct it using

$$r_d(\mathbf{x}) = \frac{(\pi_d/\pi_t)r_t(\mathbf{x})}{(\pi_d/\pi_t)r_t(\mathbf{x}) + [(1 - \pi_d)/(1 - \pi_t)][1 - r_t(\mathbf{x})]}.$$

This assumes that the fitted training probabilities are reliable. Tree probabilities may be coarse because every leaf gives the same probability to all its examples.

For each model, show five near-boundary flows before and after prior adjustment. Explain why changing a class prior and changing a decision threshold because of error costs solve different problems.

Part 6: Multiclass classification

Include benign traffic as one class and retain each attack category containing at least 100 training, 20 validation, and 20 test examples. List the categories excluded from this multiclass comparison; do not remove them from the required binary experiments. Refit Naive Bayes, logistic regression, and the decision tree on the same partitions. Use Gaussian Naive Bayes when destination port is excluded and hybrid Naive Bayes when it is included. Select tree depth from the original candidate set using multiclass validation balanced accuracy. Compare two otherwise identical feature sets:

1. all eligible behavioral flow features without destination port;
2. the same features with destination port added as a categorical variable.

Report macro- F_1 , class-wise recall, and confusion matrices, and identify the attack categories most affected by destination port. Discuss whether the gain reflects useful information about the targeted service or dependence on the particular attack scenarios in CICIDS2017.

Connection to Lab 3

Save the complete-feature logistic-regression and selected decision-tree pipelines and predictions. Lab 3 will compare the single tree with random forests and boosting trees.

Discussion and deliverables

Discuss:

1. which Naive Bayes variant was most competitive with logistic regression and the decision tree;
2. whether the complete eligible feature set helped the generative route;
3. which assumption or representation choice most limited a model;
4. what advantage the explicit class prior provided;
5. how destination port affected multiclass classification;
6. what remains specific to CICIDS2017.

Submit reproducible Python code, the representation table, required plots and results, and a 3 to 4 page report. In an appendix, list feature names, final dimensions, model settings, class counts, split sizes, and random seeds.

Assessment criteria

Criterion	Weight
Dataset preparation, representations, and random-split protocol	20%
Correct implementation of Naive Bayes models	20%
Binary and multiclass comparisons with discriminative models	25%
Assumption checks and interpretation	20%
Limited-data and prior-shift analysis	15%